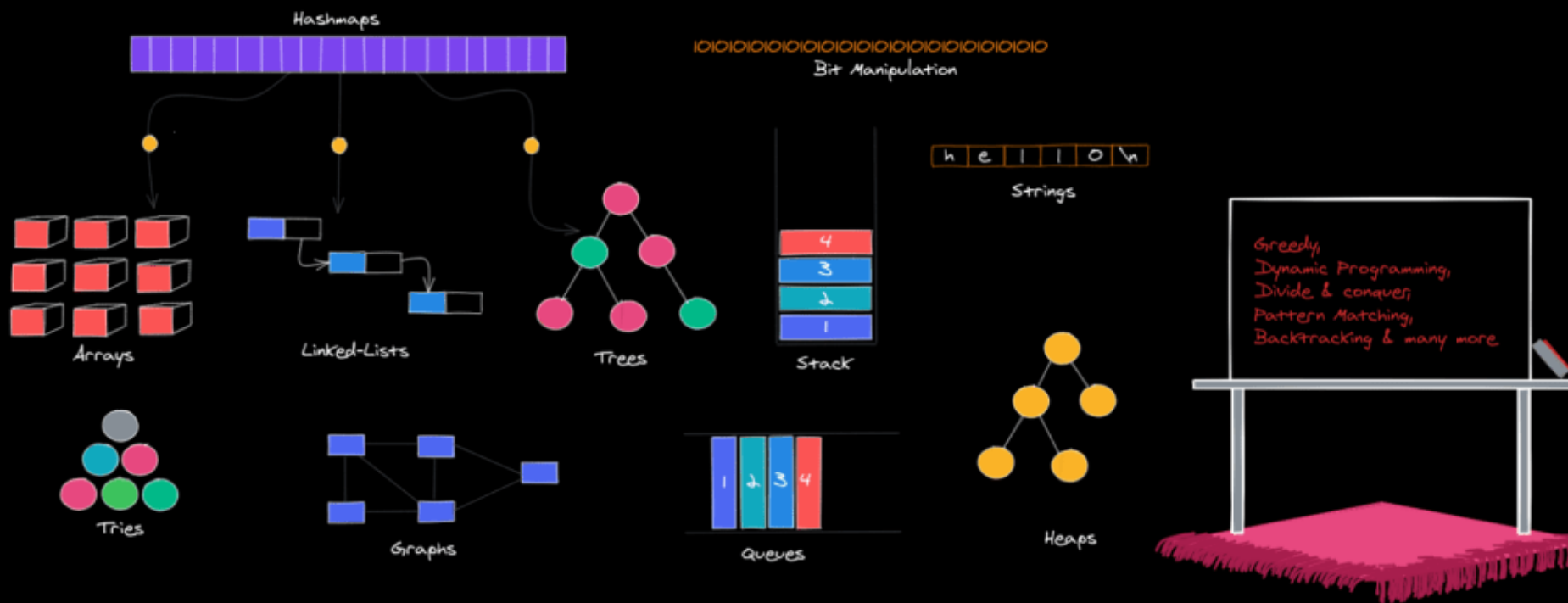




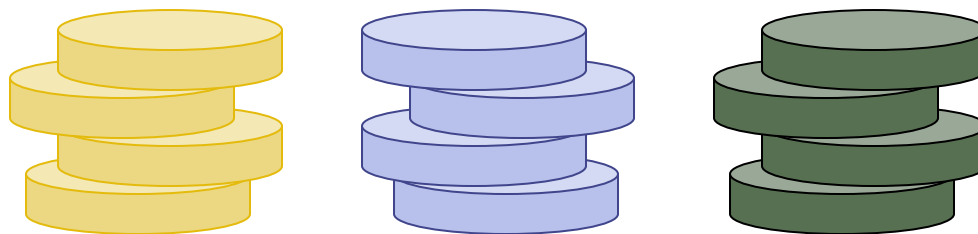
数据结构与算法导论

刘子博

FDSM

liuzibo@fudan.edu.cn

栈



栈



- 栈：限制数据插入、删除操作只能在一端进行的特殊序列（LIFO）
- 允许插入、删除的一端为栈顶（top），另一端为栈底（bottom）
- 在栈顶插入元素称为入栈（push），删除元素称为出栈（pop）
- 栈中元素个数称为栈的长度
- 栈的例子：
 - 糖果售卖器
 - 浏览器的“返回”功能
 - 文本编辑器的“撤销”功能
 -

栈的ADT

ADT Stack {

数据对象: $S = \{S_1, S_2, \dots, S_N\}$

基本操作:

S.pop(): 删除栈顶元素, 并返回其值

S.push(e): 将一个新元素e添加到栈顶

S.top(): 在不移除栈顶元素的前提下, 返回栈顶元素的值

len(S): 返回栈S的长度

S.init(S_0): 使用序列 S_0 初始化栈S

S.is_empty(): 检查栈S是否为空, 如为空则返回True

S.clear(): 清空栈S

} ADT Stack

例：栈的操作

Operation	Return Value	Stack Contents
S.push(5)	—	[5]
S.push(3)	—	[5, 3]
len(S)	2	[5, 3]
S.pop()	3	[5]
S.is_empty()	False	[5]
S.pop()	5	[]
S.is_empty()	True	[]
S.pop()	“error”	[]
S.push(7)	—	[7]
S.push(9)	—	[7, 9]
S.top()	9	[7, 9]
S.push(4)	—	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop()	4	[7, 9]
S.push(6)	—	[7, 9, 6]
S.push(8)	—	[7, 9, 6, 8]
S.pop()	8	[7, 9, 6]

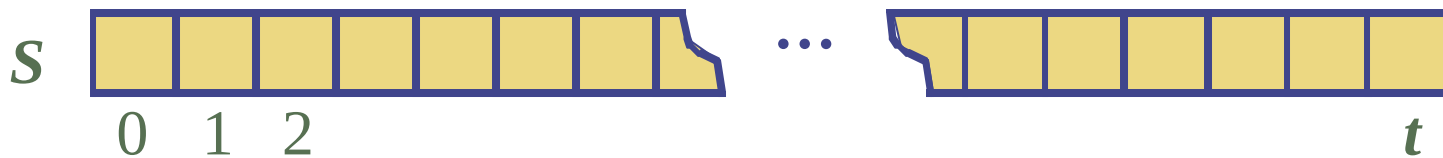
基于数组的栈实现

- 实现栈的一种简单的方法是使用数组
- 采用从左到右的方式添加元素，从右到左的方式删除元素
- 使用一个变量记录栈顶元素的位置



基于数组的栈实现

- 存储栈元素的数组可能被填满
- 这时一个入栈操作需要先将数组大小扩大，再将新元素压入栈顶



基于数组的栈实现

- 我们采用Python的list类帮助我们实现栈
- list类的一些方法不符合栈的ADT，且名称也不相符
 - 包含一个现存类（list）的实例作为私有成员，将其方法改编以实现新类

使用Python的list类实现栈

```
class ArrayStack:
    """LIFO Stack implementation using a Python list as underlying storage."""

    def __init__(self):
        """Create an empty stack."""
        self._data = []                # nonpublic list instance

    def __len__(self):
        """Return the number of elements in the stack."""
        return len(self._data)

    def is_empty(self):
        """Return True if the stack is empty."""
        return len(self._data) == 0

    def push(self, e):
        """Add element e to the top of the stack."""
        self._data.append(e)          # new item stored at end of list
```

使用Python的list类实现栈

```
def top(self):
    """Return (but do not remove) the element at the top of the stack.

    Raise Empty exception if the stack is empty.
    """
    if self.is_empty():
        raise Empty('Stack is empty')
    return self._data[-1]                # the last item in the list

def pop(self):
    """Remove and return the element from the top of the stack (i.e., LIFO).

    Raise Empty exception if the stack is empty.
    """
    if self.is_empty():
        raise Empty('Stack is empty')
    return self._data.pop()              # remove last item from list
```

性能分析

- 对于长为 n 的栈：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>S.push(e)</code>	
<code>S.pop()</code>	
<code>S.top()</code>	
<code>S.is_empty()</code>	
<code>len(S)</code>	

性能分析

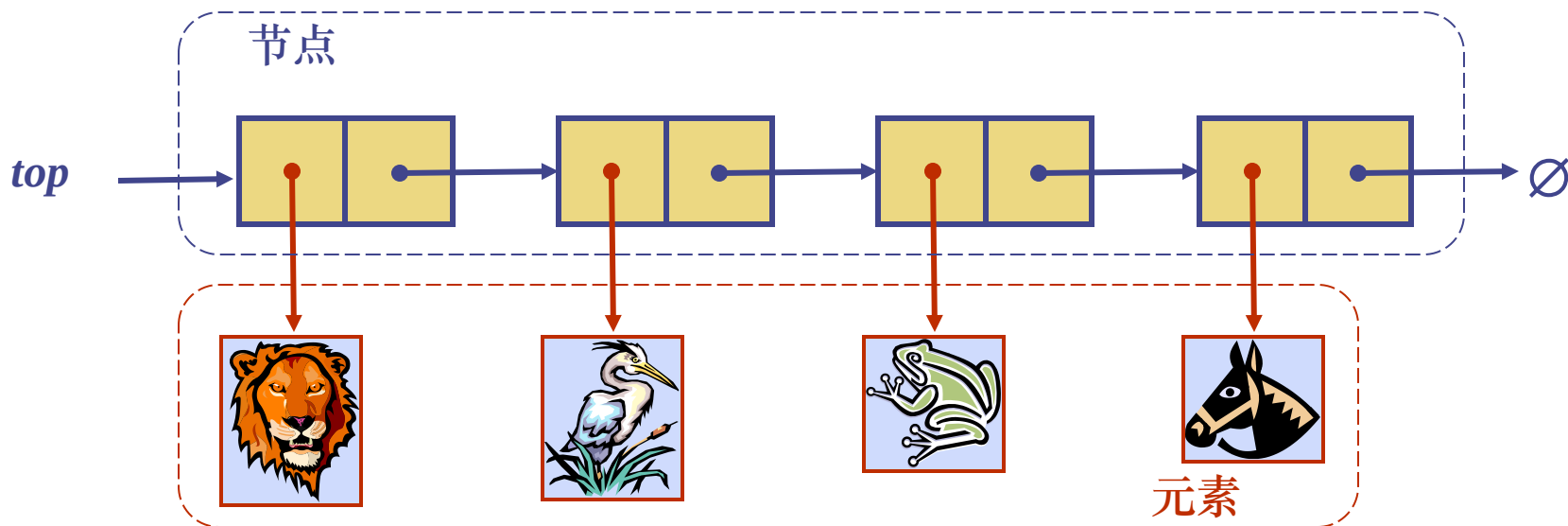
- 对于长为 n 的栈：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>S.push(e)</code>	$O(1)^*$
<code>S.pop()</code>	$O(1)^*$
<code>S.top()</code>	$O(1)$
<code>S.is_empty()</code>	$O(1)$
<code>len(S)</code>	$O(1)$

*摊销

用单向链表实现栈

- 我们可以通过单向链表实现栈
- 规定栈顶在链表的头部



用单向链表实现栈

```
class LinkedStack:
    """LIFO Stack implementation using a singly linked list for storage."""

    #----- nested _Node class -----
    class _Node:
        """Lightweight, nonpublic class for storing a singly linked node."""
        __slots__ = '_element', '_next'           # streamline memory usage

        def __init__(self, element, next):        # initialize node's fields
            self._element = element              # reference to user's element
            self._next = next                    # reference to next node

    #----- stack methods -----
    def __init__(self):
        """Create an empty stack."""
        self._head = None                       # reference to the head node
        self._size = 0                          # number of stack elements

    def __len__(self):
        """Return the number of elements in the stack."""
        return self._size
```

用单向链表实现栈

```
def is_empty(self):
    """Return True if the stack is empty."""
    return self._size == 0

def push(self, e):
    """Add element e to the top of the stack."""
    self._head = self._Node(e, self._head) # create and link a new node
    self._size += 1

def top(self):
    """Return (but do not remove) the element at the top of the stack.

    Raise Empty exception if the stack is empty.
    """
    if self.is_empty():
        raise Empty('Stack is empty')
    return self._head._element # top of stack is at head of list
```

用单向链表实现栈

```
def pop(self):  
    """Remove and return the element from the top of the stack (i.e., LIFO).  
  
    Raise Empty exception if the stack is empty.  
    """  
    if self.is_empty():  
        raise Empty('Stack is empty')  
    answer = self._head._element  
    self._head = self._head._next          # bypass the former top node  
    self._size -= 1  
    return answer
```

性能分析

- 对于长为 n 的栈：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>S.push(e)</code>	
<code>S.pop()</code>	
<code>S.top()</code>	
<code>S.is_empty()</code>	
<code>len(S)</code>	

性能分析

- 对于长为 n 的栈：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>S.push(e)</code>	$O(1)$
<code>S.pop()</code>	$O(1)$
<code>S.top()</code>	$O(1)$
<code>S.is_empty()</code>	$O(1)$
<code>len(S)</code>	$O(1)$



栈的应用

- 直接应用
 - 网页浏览器的历史记录
 - 文本编辑器的撤销键入
 - 支持递归的语言中的函数调用链
- 间接应用
 - 一些算法的辅助数据结构
 - 其他数据结构的一部分

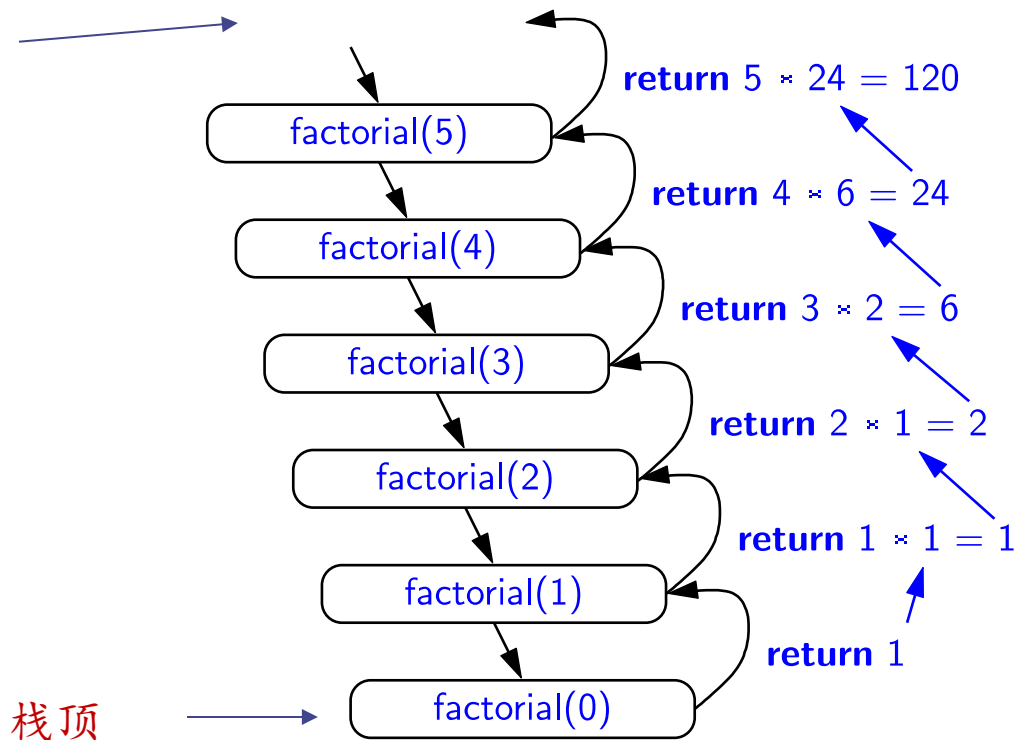
函数调用

- 当调用新函数时，系统需要保存当前调用的所有局部变量，否则新函数将覆盖这些变量
- 此外，必须保存例程中的当前位置，以便新功能在完成后知道返回何处
- 所有这些工作都可以通过操作系统自动使用栈来完成
- 在内存中，**递归**实际上是通过**栈**操作来完成的

函数调用

- 例：阶乘函数的递归调用

栈底



函数的递归调用

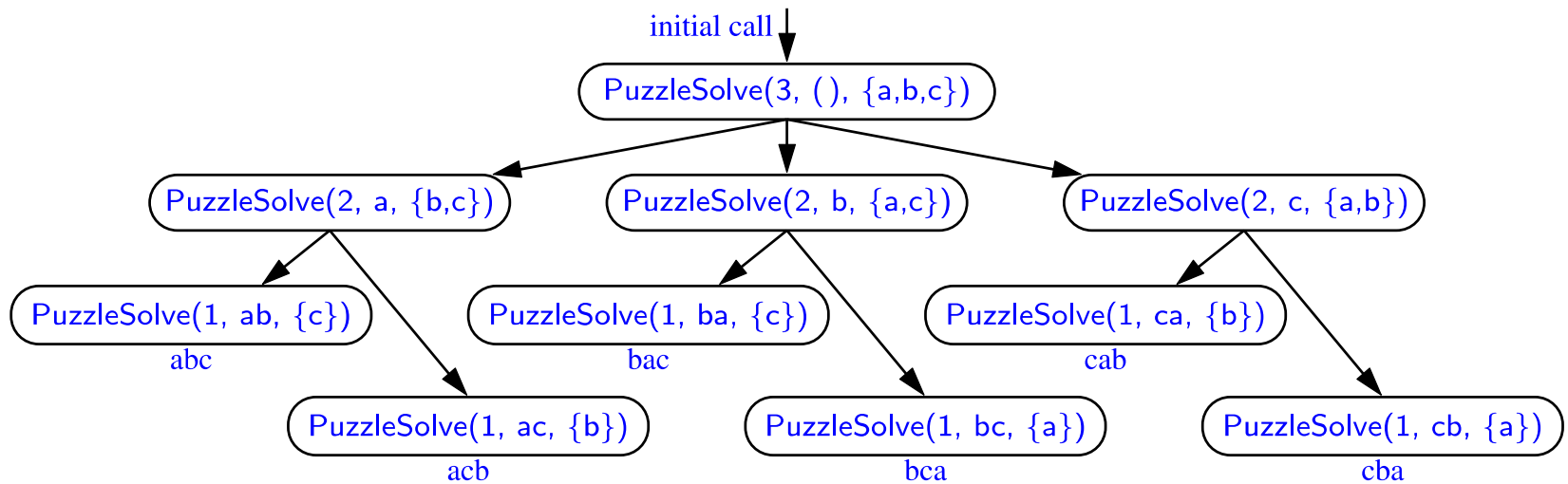
- 很多时候，栈主要用来实现回溯功能，而回溯功能是递归的主要特点之一
- 因此很多情况下，基于栈的操作可以实现与递归同样的效果
- 在深度优先搜索（Depth-First-Search）中（简称为DFS），常常使用到栈及递归的思想

回溯法

- **回溯法** (backtracking) 是穷举法 (或称为蛮力法、暴力搜索法) 中的一种
- 回溯法沿用栈的思想
- 对于合适的问题, 使用回溯法可以找到所有解, 必要时需要进行 **剪枝** (pruning) 避免对解进行无效的搜索
- 与递归类似, 回溯法可分解为很多步, 每一步的常见做法为:
 - 首先从该步可以尝试的所有元素的集合 U 中选出一个元素 e
 - 将该元素 e 加入到当前尝试的解中, 进入下一步
 - ◆ 在某些条件下可以在该步判定当前尝试的解是否为真解, 以及当前情况可否进行剪枝
 - 将元素 e 从当前解中移除 (有必要时需将 e 放回集合 U)

回溯法

- 回溯法例子：求和谜题
- 回溯法常用的辅助数据结构：树



使用栈进行数据逆置

- 栈可以用来实现一个数据序列的逆置:

```
def reverse_file(filename):  
    """Overwrite given file with its contents line-by-line reversed."""  
    S = ArrayStack()  
    original = open(filename)  
    for line in original:  
        S.push(line.rstrip('\n'))    # we will re-insert newlines when writing  
    original.close()  
  
    # now we overwrite with contents in LIFO order  
    output = open(filename, 'w')    # reopening file overwrites original  
    while not S.is_empty():  
        output.write(S.pop() + '\n') # re-insert newline characters  
    output.close()
```

括号匹配

- 每一个左括号 “(” , “{” , 或 “[” 必须与对应的右括号 “)” , “}” , 或 “]” 匹配, 例如:
 - 正确: `() (()) { ([()] ) }`
 - 正确: `((() (()) { ([()] ) } ) )`
 - 错误: `) (()) { ([()] ) }`
 - 错误: `( { [ ] ) }`
 - 错误: `(`

括号匹配算法

Algorithm ParentMatch(X, n):

Input: An array X of n tokens, each of which is either a grouping symbol, a variable, an arithmetic operator, or a number

Output: **true** if and only if all the grouping symbols in X match

Let S be an empty stack

for $i=0$ to $n-1$ **do**

if $X[i]$ is an opening grouping symbol **then**

$S.push(X[i])$

else if $X[i]$ is a closing grouping symbol **then**

if $S.is_empty()$ **then**

return false {nothing to match with}

if $S.pop()$ does not match the type of $X[i]$ **then**

return false {wrong type}

if $S.isEmpty()$ **then**

return true {every symbol matched}

else return false {some symbols were never matched}

括号匹配算法

```
def is_matched(expr):  
    """Return True if all delimiters are properly match; False otherwise."""  
    lefty = '({['          # opening delimiters  
    righty = ')}]}'        # respective closing delimiters  
    S = ArrayStack()  
    for c in expr:  
        if c in lefty:  
            S.push(c)        # push left delimiter on stack  
        elif c in righty:  
            if S.is_empty():  
                return False    # nothing to match with  
            if righty.index(c) != lefty.index(S.pop()):  
                return False    # mismatched  
    return S.is_empty()      # were all symbols matched?
```

HTML标签匹配

- 在标记语言（如HTML或XML）的验证中，需要对分隔符匹配进行测试
- HTML文本中，部分文本由HTML标签分隔。如一个HTML开始标签的形式为“<name>”，则其相应结束标签的形式则为“</name>”

HTML标签匹配

- 例:

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

计算算数表达式

$$14 - 3 * 2 + 7 = (14 - (3 * 2)) + 7$$

运算符优先级:

* 优先级比 + / - 高

运算符结合规律:

同一优先级的运算符从左往右运算

例: $(x - y) + z$ 而非 $x - (y + z)$

算法设计思路: 将运算符依次放入栈中, 出栈时先弹出优先级更高的或相同优先级的运算符

计算算数表达式算法

建立两个栈:

- opStk栈保存运算符
- valStk栈保存值
- 使用\$作为输入结束的运算符, 其优先级最低

Algorithm doOp()

```
x ← valStk.pop();  
y ← valStk.pop();  
op ← opStk.pop();  
valStk.push( y op x )
```

Algorithm repeatOps(refOp):

```
while (valStk.size() > 1 ∧  
      prec(refOp) ≤  
      prec(opStk.top()))  
  doOp()
```

Algorithm EvalExp()

Input: a stream of tokens representing an arithmetic expression (with numbers)

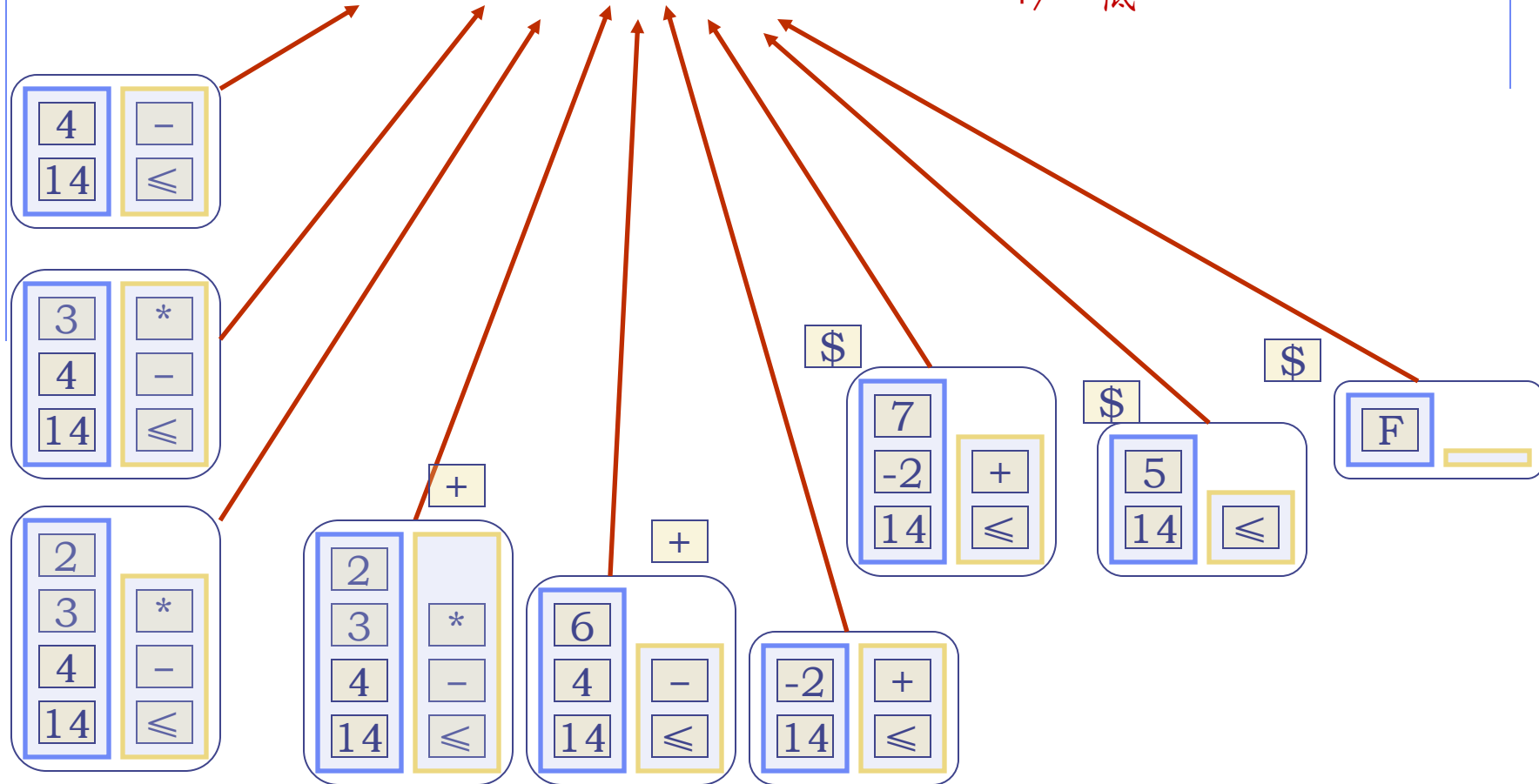
Output: the value of the expression

```
while there's another token z  
  if isNumber(z) then  
    valStk.push(z)  
  else  
    repeatOps(z);  
    opStk.push(z)  
repeatOps($);  
return valStk.top()
```

计算算数表达式

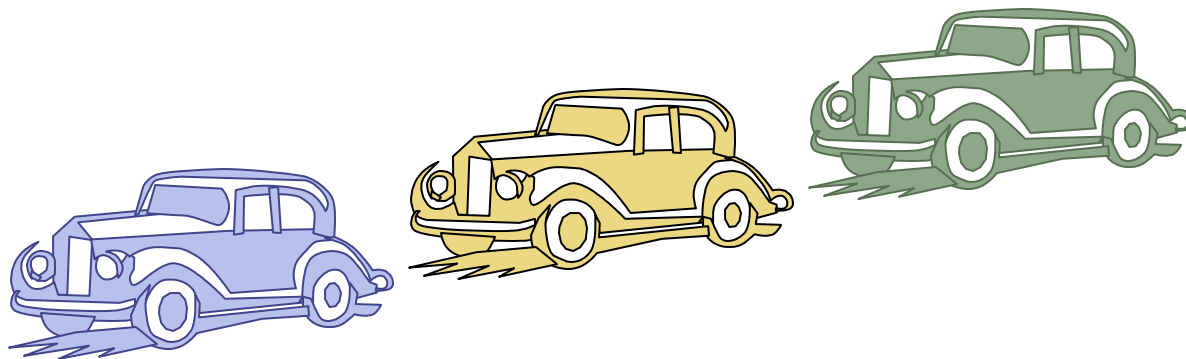
$$14 \leq 4 - 3 * 2 + 7$$

运算符 $\leq$ 优先级比
+/- 低



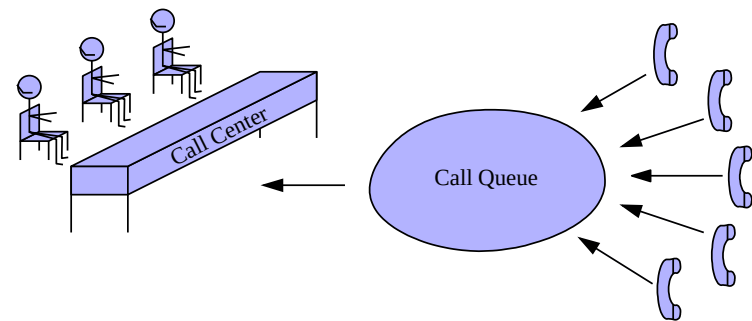
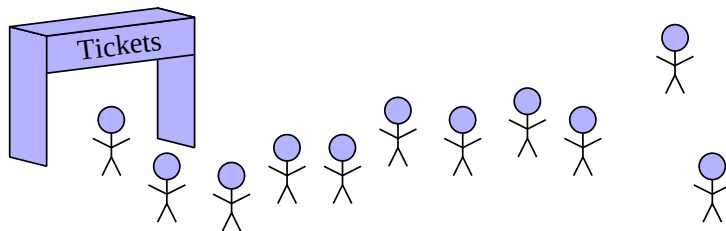


队列



队列

- 队列：限制数据插入在一端进行、删除在另一端进行的特殊序列 (FIFO)
- 允许插入的一端称为**队尾**，允许删除的一端称为**队头**
- 在队尾插入元素称为**入队 (enqueue)**，在队头删除元素称为**出队 (dequeue)**
- 队列中元素个数称为队列的长度



队列的ADT

ADT Queue {

数据对象: $Q = \{Q_1, Q_2, \dots, Q_N\}$

基本操作:

Q.enqueue(e): 向队列Q的队尾添加一个元素

Q.dequeue(): 从队列Q中移除并返回第一个元素

Q.first(): 在不移除的前提下, 返回队列的第一个元素

len(Q): 返回队列Q的长度

Q.init(Q₀): 使用序列Q₀初始化队列Q

Q.is_empty(): 检查队列Q是否为空, 如为空则返回True

Q.clear(): 清空队列Q

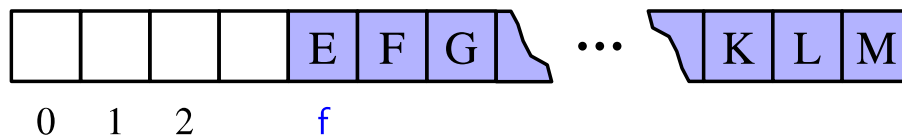
} ADT Queue

例：队列的操作

Operation	Return Value	first $\leftarrow$ Q $\leftarrow$ last
Q.enqueue(5)	—	[5]
Q.enqueue(3)	—	[5, 3]
len(Q)	2	[5, 3]
Q.dequeue()	5	[3]
Q.is_empty()	False	[3]
Q.dequeue()	3	[]
Q.is_empty()	True	[]
Q.dequeue()	“error”	[]
Q.enqueue(7)	—	[7]
Q.enqueue(9)	—	[7, 9]
Q.first()	7	[7, 9]
Q.enqueue(4)	—	[7, 9, 4]
len(Q)	3	[7, 9, 4]
Q.dequeue()	7	[9, 4]

基于数组的队列实现

- 可以使用数组的方式来实现队列
- 潜在的问题：append(e)高效，但pop(0)很低效
- 解决方案：用一个变量f存储当前队头元素的索引，用指代为空的指针代替数组中离队的元素
- 缺点：存储队列的列表长度为 $O(m)$ ，其中m为自队列创建以来追加元素操作的数量总和，可能远远大于队列长度



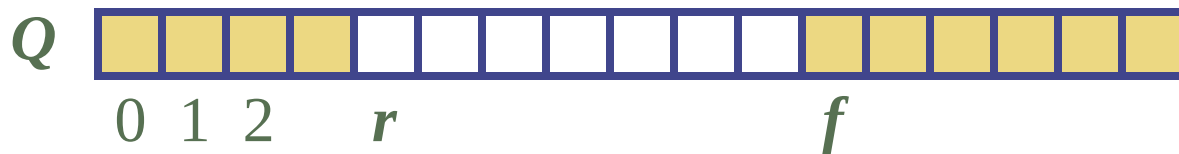
基于数组的队列实现

- 假定底层数组长度为固定值 N ，循环使用数组
- 用两个变量分别记录队头和队尾的位置

f 记录队头位置

r 记录队尾之后的第一个位置，即下一个元素插入的位置

- f 和 r 到达数组末端后返回数组前端



队列操作

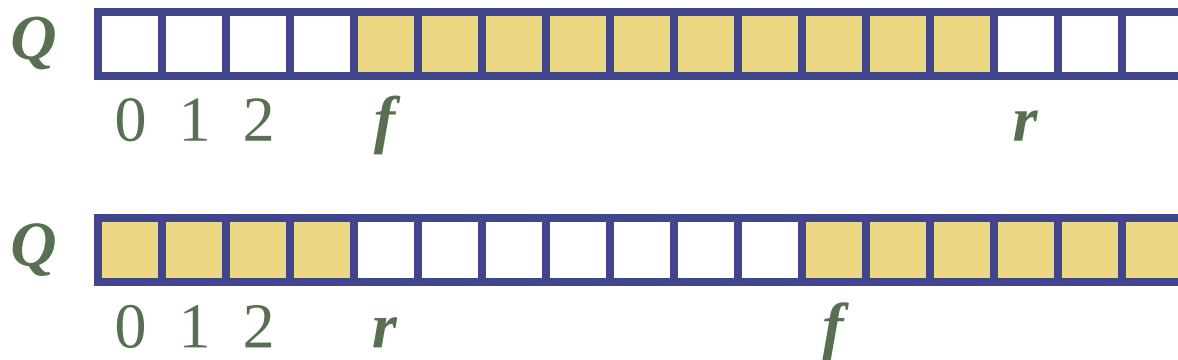
- 使用模运算（求余数）对队头和队尾索引进行操作
- 潜在问题？

Algorithm *size()*

return $(r - f + N) \bmod N$

Algorithm *isEmpty()*

return $(f = r)$

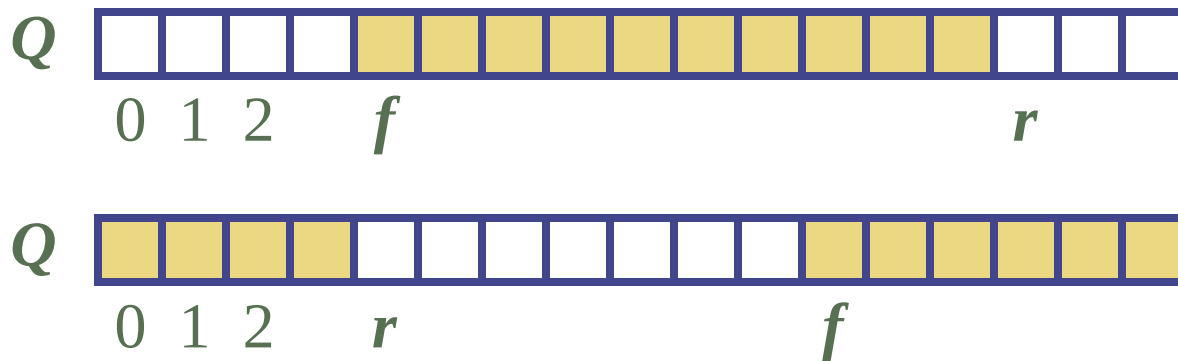


队列操作

- 队列的入队操作:

```

Algorithm enqueue(o)
  if  $\text{size}() = N - 1$  then
    throw FullQueueException
  else
     $Q[r] \leftarrow o$ 
     $r \leftarrow (r + 1) \bmod N$ 
  
```

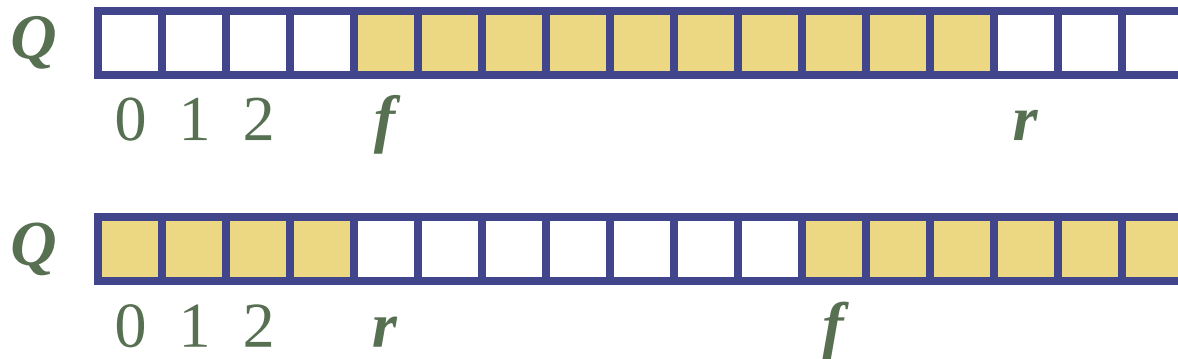


队列操作

- 队列的出队操作:

```

Algorithm dequeue()
  if isEmpty() then
    throw EmptyQueueException
  else
     $o \leftarrow Q[f]$ 
     $f \leftarrow (f + 1) \bmod N$ 
    return  $o$ 
  
```

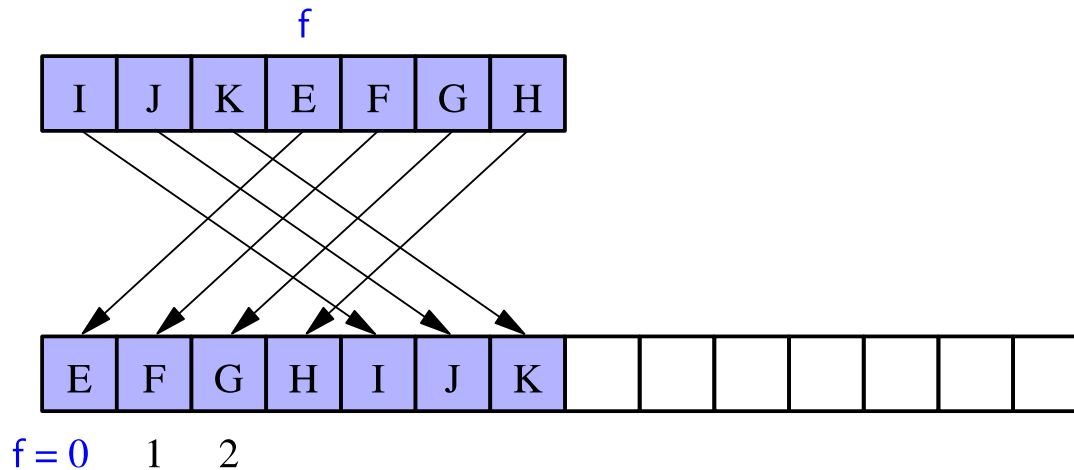


基于数组的队列实现方法

- 我们使用如下3个变量维护队列：
 - `_data`: 一个长度（容量）固定的list对象
 - `_size`: 一个整数，代表当前队列内存储的元素数量
 - `_front`: 一个整数，代表`_data`实例list中队列第一个元素的位置
- 这样的存储方式避免了 $f = r$ 时空队列与满队列之间的混淆

基于数组的队列实现方法

- 当队列大小与底层_data大小相等时，再插入元素即需扩大底层_data的大小，我们可采取倍增底层列表长度的方式
- 将队列的元素从原底层数组转移到新底层数组时，将队列头索引初始化为0，以免队列索引出现错误



基于数组的队列实现方法

```
class ArrayQueue:
    """FIFO queue implementation using a Python list as underlying storage."""
    DEFAULT_CAPACITY = 10          # moderate capacity for all new queues

    def __init__(self):
        """Create an empty queue."""
        self._data = [None] * ArrayQueue.DEFAULT_CAPACITY
        self._size = 0
        self._front = 0

    def __len__(self):
        """Return the number of elements in the queue."""
        return self._size

    def is_empty(self):
        """Return True if the queue is empty."""
        return self._size == 0
```

基于数组的队列实现方法

```
def first(self):
    """Return (but do not remove) the element at the front of the queue.

    Raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is empty')
    return self._data[self._front]

def dequeue(self):
    """Remove and return the first element of the queue (i.e., FIFO).

    Raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is empty')
    answer = self._data[self._front]
    self._data[self._front] = None          # help garbage collection
    self._front = (self._front + 1) % len(self._data)
    self._size -= 1
    return answer
```

基于数组的队列实现方法

```
def enqueue(self, e):
    """Add an element to the back of queue."""
    if self._size == len(self._data):
        self._resize(2 * len(self._data))    # double the array size
    avail = (self._front + self._size) % len(self._data)
    self._data[avail] = e
    self._size += 1

def _resize(self, cap):
    """Resize to a new list of capacity >= len(self)."""
    old = self._data
    self._data = [None] * cap
    walk = self._front
    for k in range(self._size):
        self._data[k] = old[walk]
        walk = (1 + walk) % len(old)
    self._front = 0
```

we assume cap >= len(self)
keep track of existing list
allocate list with new capacity
only consider existing elements
intentionally shift indices
use old size as modulus
front has been realigned

性能分析

- 对于长为 n 的队列：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>Q.enqueue(e)</code>	
<code>Q.dequeue()</code>	
<code>Q.first()</code>	
<code>Q.is_empty()</code>	
<code>len(Q)</code>	

性能分析

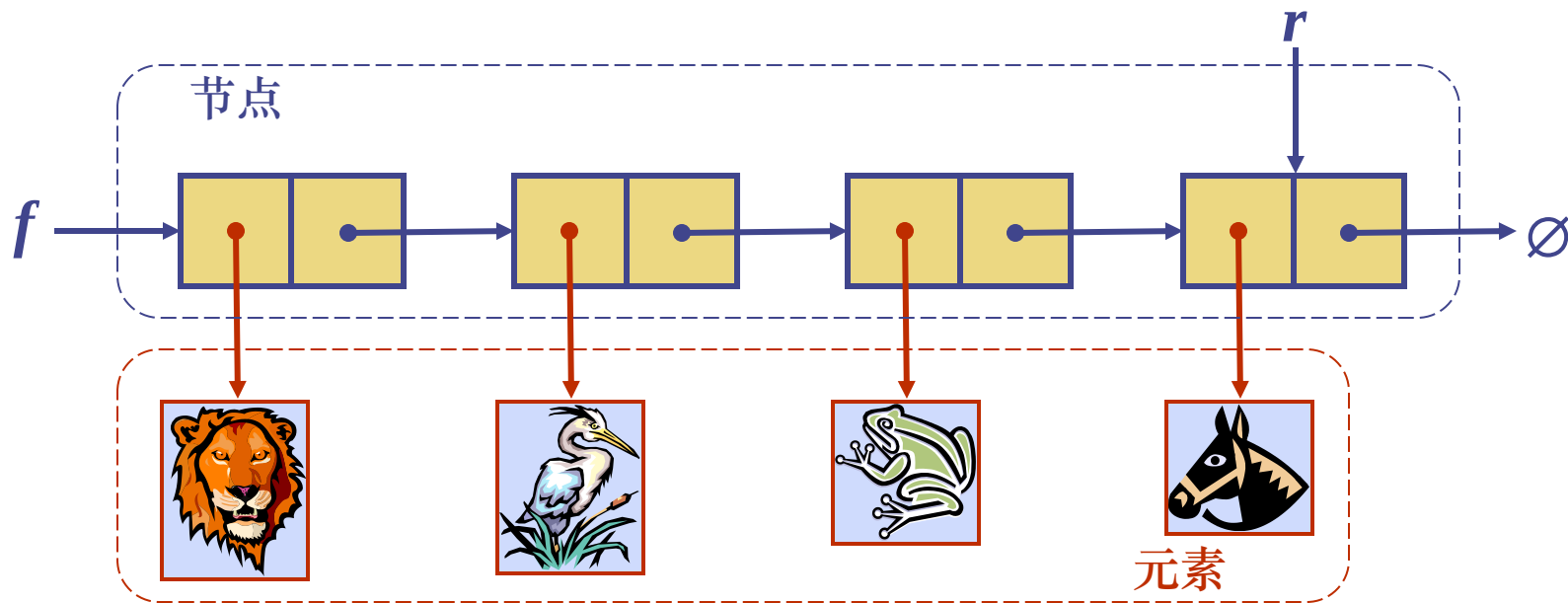
- 对于长为 n 的队列：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>Q.enqueue(e)</code>	$O(1)^*$
<code>Q.dequeue()</code>	$O(1)^*$
<code>Q.first()</code>	$O(1)$
<code>Q.is_empty()</code>	$O(1)$
<code>len(Q)</code>	$O(1)$

*摊销

用单向链表实现队列

- 我们可以使用单向链表实现队列
- 队列的队头存储在链表头部，队尾存储在链表尾部



用单向链表实现队列

```
class LinkedQueue:
    """FIFO queue implementation using a singly linked list for storage."""

    #----- nested _Node class -----
    class _Node:
        """Lightweight, nonpublic class for storing a singly linked node."""
        __slots__ = '_element', '_next'           # streamline memory usage

        def __init__(self, element, next):        # initialize node's fields
            self._element = element               # reference to user's element
            self._next = next                     # reference to next node

    #----- queue methods -----
    def __init__(self):
        """Create an empty queue."""
        self._head = None
        self._tail = None
        self._size = 0                            # number of queue elements

    def __len__(self):
        """Return the number of elements in the queue."""
        return self._size
```

用单向链表实现队列

```
def is_empty(self):
    """Return True if the queue is empty."""
    return self._size == 0

def first(self):
    """Return (but do not remove) the element at the front of the queue.

    Raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is empty')
    return self._head._element # front aligned with head of list
```

用单向链表实现队列

```
def dequeue(self):
    """Remove and return the first element of the queue (i.e., FIFO).

    Raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is empty')
    answer = self._head._element
    self._head = self._head._next
    self._size -= 1
    if self.is_empty():
        self._tail = None
    return answer

def enqueue(self, e):
    """Add an element to the back of queue."""
    newest = self._Node(e, None)
    if self.is_empty():
        self._head = newest
    else:
        self._tail._next = newest
    self._tail = newest
    self._size += 1
```

special case as queue is empty
removed head had been the tail

node will be new tail node

special case: previously empty

update reference to tail node

性能分析

- 对于长为 n 的队列：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>Q.enqueue(e)</code>	
<code>Q.dequeue()</code>	
<code>Q.first()</code>	
<code>Q.is_empty()</code>	
<code>len(Q)</code>	

性能分析

- 对于长为 n 的队列：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>Q.enqueue(e)</code>	$O(1)$
<code>Q.dequeue()</code>	$O(1)$
<code>Q.first()</code>	$O(1)$
<code>Q.is_empty()</code>	$O(1)$
<code>len(Q)</code>	$O(1)$

ANY
QUESTIONS
?

www.rekrutitn.com

ReKrutitn.com

队列的应用

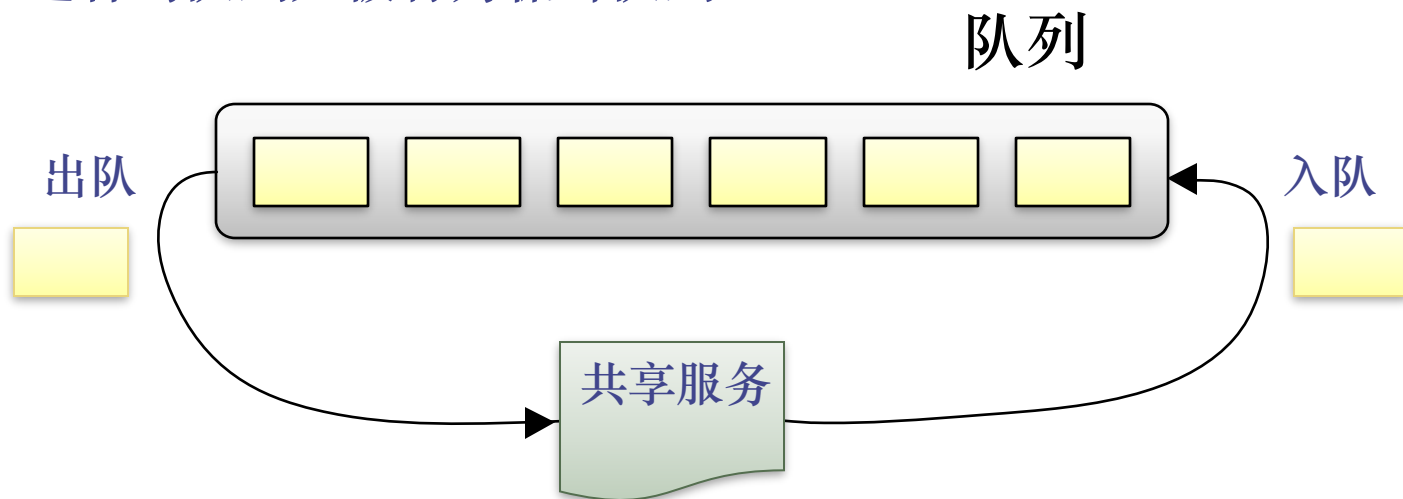
- 直接应用
 - 排队
 - 共享资源的使用（如公共打印机）
 - 多道程序（multiprogramming）、多进程、多线程
- 间接应用
 - 一些算法的辅助数据结构
 - 其他数据结构的一部分

队列的应用

- 与栈类似，队列可以作为一种辅助数据结构实现遍历
- 需要进行穷举时，也可以使用队列（使用栈则为回溯法）
- 在广度优先搜索（Breadth-First-Search）中（简称为BFS），常常使用到队列的思想

循环调度

- 可以使用一个队列Q，通过重复如下步骤，实现循环调度：
 1. $e = Q.dequeue()$
 2. Service element e
 3. $Q.enqueue(e)$
- 循环调度常用于实现计算机上多个程序并行运行
- 这样的队列又被称为循环队列



用循环链表实现循环队列

```
class CircularQueue:
    """Queue implementation using circularly linked list for storage."""

    #-----
    # nested _Node class
    class _Node:
        """Lightweight, nonpublic class for storing a singly linked node."""
        __slots__ = '_element', '_next'           # streamline memory usage

        def __init__(self, element, next):
            self._element = element
            self._next = next

    # end of _Node class
    #-----

    def __init__(self):
        """Create an empty queue."""
        self._tail = None                # will represent tail of queue
        self._size = 0                   # number of queue elements
```

用循环链表实现循环队列

```
def __len__(self):
    """Return the number of elements in the queue."""
    return self._size

def is_empty(self):
    """Return True if the queue is empty."""
    return self._size == 0

def first(self):
    """Return (but do not remove) the element at the front of the queue.

    Raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is empty')
    head = self._tail._next
    return head._element
```

用循环链表实现循环队列

```
def dequeue(self):
    """Remove and return the first element of the queue (i.e., FIFO).

    Raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is empty')
    oldhead = self._tail._next
    if self._size == 1:
        self._tail = None
    else:
        self._tail._next = oldhead._next
    self._size -= 1
    return oldhead._element
```

removing only element
queue becomes empty

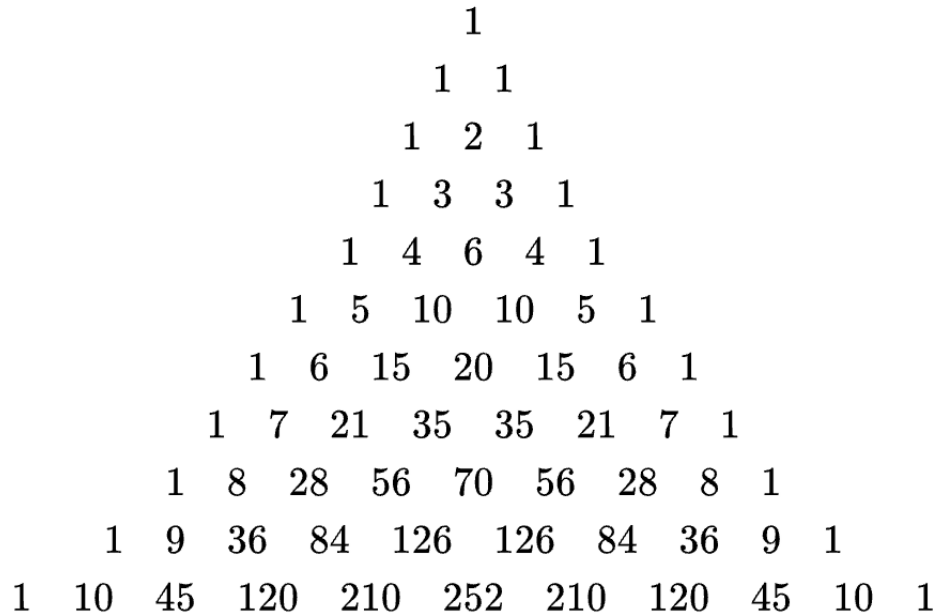
bypass the old head

用循环链表实现循环队列

```
def enqueue(self, e):  
    """Add an element to the back of queue."""  
    newest = self._Node(e, None)           # node will be new tail node  
    if self.is_empty():  
        newest._next = newest               # initialize circularly  
    else:  
        newest._next = self._tail._next   # new node points to head  
        self._tail._next = newest         # old tail points to new node  
    self._tail = newest                   # new node becomes the tail  
    self._size += 1  
  
def rotate(self):  
    """Rotate front element to the back of the queue."""  
    if self._size > 0:  
        self._tail = self._tail._next    # old head becomes new tail
```

杨辉三角

- 计算杨辉三角的第1~n行:



杨辉三角算法

Algorithm YanghuiTri(n):

Input: Number of rows n

Output: Calculate and print 1~ n rows of the Yanghui Triangle

Let Q be an empty queue

$Q.enqueue(1)$; Print '1' as the first line

for $i = 2$ to n **do**

$Q.enqueue(1)$; Print '1'

$p = Q.dequeue()$

for $k = 2$ to $i - 1$ **do**

$q = Q.dequeue()$

$Q.enqueue(p + q)$; Print ' $p + q$ '

$p = q$

$Q.enqueue(1)$; Print '1', finish this line

杨辉三角算法

```
def YanghuiTri(n):
    TriQ = ArrayQueue()
    # First line
    TriQ.enqueue(1)
    print('1'.center(5 * n))
    for m in range(2, n + 1):    # The m-th line
        Line = '1'
        TriQ.enqueue(1)
        p = TriQ.dequeue()
        for k in range(2, m):
            q = TriQ.dequeue()
            TriQ.enqueue(p + q)
            Line += ' ' + str(p + q)
        p = q
        TriQ.enqueue(1)
        Line += ' ' + '1'
        print(Line.center(5 * n))
    return
```

双端队列

- 双端队列：一种类队列数据结构，支持在队列的头部和尾部都进行插入和删除
- 双端队列被称为deque
- 例：餐馆排队时，第一位顾客进入餐馆发现没有位置，回来时会被插入到第一位

双端队列的ADT

ADT Deque {

数据对象: $D = \{D_1, D_2, \dots, D_N\}$

基本操作:

D.add_first(e): 向双端队列D的队头添加一个元素

D.add_last(e): 向队列D的队尾添加一个元素

D.delete_first(): 从双端队列Q中移除并返回第一个元素

D.delete_last(): 从双端队列Q中移除并返回最后一个元素

D.first(): 在不移除的前提下, 返回双端队列的第一个元素

D.last(): 在不移除的前提下, 返回双端队列的最后一个元素

len(D): 返回双端队列D的长度

D.init(D_0): 使用序列 D_0 初始化队列D

D.is_empty(): 检查双端队列D是否为空, 如为空则返回True

D.clear(): 清空队列D

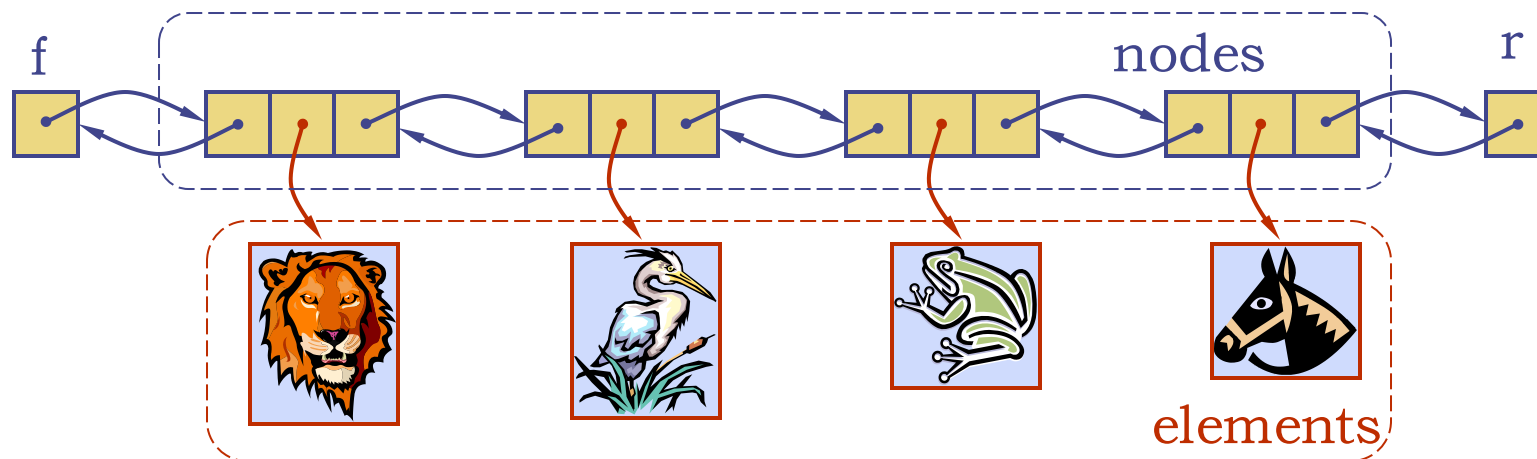
} ADT Deque

例：双端队列的操作

Operation	Return Value	Deque
D.add_last(5)	—	[5]
D.add_first(3)	—	[3, 5]
D.add_first(7)	—	[7, 3, 5]
D.first()	7	[7, 3, 5]
D.delete_last()	5	[7, 3]
len(D)	2	[7, 3]
D.delete_last()	3	[7]
D.delete_last()	7	[]
D.add_first(6)	—	[6]
D.last()	6	[6]
D.add_first(8)	—	[8, 6]
D.is_empty()	False	[8, 6]
D.last()	6	[8, 6]

用双向链表实现双端队列

- 我们可以使用双向链表实现双端队列



用双向链表实现双端队列

```
class LinkedDeque(DoublyLinkedList):          # note the use of inheritance
    """Double-ended queue implementation based on a doubly linked list."""

    def first(self):
        """Return (but do not remove) the element at the front of the deque.

        Raise Empty exception if the deque is empty.
        """
        if self.is_empty():
            raise Empty("Deque is empty")
        return self._header._next._element    # real item just after header

    def last(self):
        """Return (but do not remove) the element at the back of the deque.

        Raise Empty exception if the deque is empty.
        """
        if self.is_empty():
            raise Empty("Deque is empty")
        return self._trailer._prev._element  # real item just before trailer
```

用双向链表实现双端队列

```
def insert_first(self, e):  
    """Add an element to the front of the deque."""  
    self._insert_between(e, self._header, self._header._next)    # after header  
  
def insert_last(self, e):  
    """Add an element to the back of the deque."""  
    self._insert_between(e, self._trailer._prev, self._trailer) # before trailer
```

用双向链表实现双端队列

```
def delete_first(self):
    """Remove and return the element from the front of the deque.

    Raise Empty exception if the deque is empty.
    """
    if self.is_empty():
        raise Empty("Deque is empty")
    return self.delete(self._header._next) # use inherited method

def delete_last(self):
    """Remove and return the element from the back of the deque.

    Raise Empty exception if the deque is empty.
    """
    if self.is_empty():
        raise Empty("Deque is empty")
    return self.delete(self._trailer._prev) # use inherited method
```

性能分析

- 对于长为 n 的双端队列：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>L.first()</code>	
<code>L.last()</code>	
<code>L.insert_first(e)</code>	
<code>L.insert_last(e)</code>	
<code>L.delete_first()</code>	
<code>L.delete_last()</code>	

性能分析

- 对于长为 n 的双端队列：
 - 其空间利用率为 $O(n)$
 - 其各操作的时间复杂度为：

操作	运行时间
<code>L.first()</code>	$O(1)$
<code>L.last()</code>	$O(1)$
<code>L.insert_first(e)</code>	$O(1)$
<code>L.insert_last(e)</code>	$O(1)$
<code>L.delete_first()</code>	$O(1)$
<code>L.delete_last()</code>	$O(1)$



作业1

- 第一次作业已在elearning发布
- 每次作业100分，在期末成绩中折算为10分
- 作业截止日期为两周后的周二晚22:00
- 作业截止后至当周周三上课前提交的作业视为迟交作业，扣20分起评
- 两周后的周三上课后提交的作业按0分计

Enjoy